~~Network~~ Benchmarking Methodology Working Group
V. Vassilev
Internet-Draft Lightside Instruments AS
Intended status: Standards Track 7 April 2026
Expires: 9 October 2026

A YANG Data Model for Network Tester Management
draft-ietf-bmwg-network-tester-cfg-10

Abstract

This document ~~introduces~~ specifies a ~~new~~ YANG data model for use in network interconnect testing setups that ~~-~~contain~~ing~~ ~~modules~~ instances of traffic generator and traffic analyzer.

Commenté [MB1]: Please refer to Section 2.5 of RFC9907 on the guidance about model, data model, etc.

Commenté [MB2]: To avoid confusion with yang module.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at https://datatracker.ietf.org/drafts/current/.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 9 October 2026.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (https://trustee.ietf.org/license-info) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document. Code Components extracted from this document must include Revised BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Revised BSD License.

Table of Contents

1. Introduction 2
  1.1. Terminology 2
    1.1.1. Definitions and Acronyms 2

1.1.2. Tree Diagram 3
1.2. Problem Statement 3
1.3. Objectives 3
1.4. Solution 4
2. Using the network interconnect tester model 5
3. Traffic Generator Module Tree Diagram 5
4. Traffic Analyzer Module Tree Diagram 6
5. Traffic Generator Module YANG 8
6. Traffic Analyzer Module YANG 15
7. IANA Considerations 22
7.1. URI Registration 23
7.2. YANG Module Name Registration 23
8. Security Considerations 23
8.1. ietf-traffic-generator.yang 24
8.2. ietf-traffic-analyzer.yang 24
9. References 24
9.1. Normative References 24
9.2. Informative References 25
Appendix A. Examples 25
A.1. Basic Test Program 26
A.2. Generating RFC2544 Testframes 27
A.3. Generating frames with dynamic data fields 27
A.4. Example: NETCONF <get-config> Reply 27
Author's Address 28

1. Introduction

There is a need for standard mechanism to allow the specification and implementation of the transactions part of network tests. ~~The~~Such a mechanism should allow the control and monitoring of the data plane traffic in a transactional ~~way~~manner. This document defines a YANG Data model which is composed of two YANG [RFC7950] modules for test traffic generator and analyzer functions.

~~The YANG modules in this document conform to the Network Management Datastore Architecture (NMDA) defined in RFC 8342.~~

1.1. Terminology

1.1.1. Definitions and Acronyms

DUT: Device Under Test

TA: Traffic Analyzer

TG: Traffic Generator

1.1.2. Tree Diagram

The meanings of the symbols in the YANG tree diagrams are defined in ~~For a reference to the annotations used in tree diagrams included in this document, please see YANG Tree Diagrams~~ [RFC8340].

1.2. Problem Statement

Network interconnect tests require active network elements part of the tested network that generate test traffic and network elements

Commenté [MB3]: What does that mean concretely? Can we please clarify that in the text to have self-contained intro?

Commenté [MB4]: This is not required anymore in RFC9907.

A mention is needed only when there are major deviations.

Commenté [MB5]: There are several terms used in the document that need some defs: stream, modifiers, etc.

Commenté [MB6]: Please add references for where an authoritative definition of these terms can be found + list those as normative.

If not, then please add a definition of each of the terms.

Commenté [MB7]: Move to a separate section after the introduction

that analyze the test traffic at one or more points of its forwarding path. A network interconnect tester is ~~a device~~an entity that can either generate test traffic, analyze test traffic, or both. ~~Here is a f~~Figure 1, borrowed from [RFC2544], represent~~ing~~ the horseshoe test setup topology consisting of a single tester and a single Device Under Test (DUT) connected in a network interconnect loop.

```
                  +------------+
                  |            |
      +-----------|  tester    |<-------------+
      |           |            |              |
      |           +------------+              |
      |                                       |
      |           +------------+              |
      |           |            |              |
      +---------->|    DUT     |--------------+
                  |            |
                  +------------+
```

Figure 1: TBC

This document ~~attempts to address the problem of defining~~ defines a YANG data model of a network interconnect tester that can be used for development of ~~vendor~~ vendor-independent network interconnect tests and utilize the advantages of transactional management using standard protocols like Network Configuration Protocol (NETCONF) [RFC6241] and RESTCONF [RFC8040]~~NETCONF~~.

1.3. Objectives

This section specifies the design objectives for the data model. It should:

* provide means to specify the generated traffic as streams of cyclic sequence of bursts with configurable frame size, frame data, inter-frame gap, and inter-burst gap.

* have a mandatory single stream mode and optional multi stream mode.

* provide means for configuration of traffic streams with static frame data where frames with identical frame data are sent during the lifetime of ~~the~~ a stream.

* provide means for configuration of traffic streams with dynamic frame data where frames contain fields with dynamic data like generation time and sequence number.

* allow third parties to augment the base modules with alternative dynamic fields of frame data extensions.

* provide means for realtime synchronization and orchestration of the generated streams.

Commenté [MB8]: To cover virtual instances

Commenté [MB9]: Please add caption/Legend for all figures

Commenté [MB10]: Readers may need to understand the reasoning of why this is required. Unless this is elaborated, please consider add a reference to back the requirement. The same apply for the other items. Thanks.

Commenté [MB11]: To be consistent with Section 3.7 of rfc1242

Commenté [MB12]: That is? We need at least a reference.

a mis en forme : Anglais (États-Unis)

Commenté [MB13]: As you have two

Commenté [MB14]: Modules can be always augmented. You may consider which part of the design is a provision to meet this.

* provide counters for received test traffic frames and octets.

* provide latency statistics in the case of test traffic with dynamic frame data that includes timestamp.

* provide sequence number errors in the case of test traffic with dynamic frame data that includes sequence numbers.

* provide means for capturing traffic frames data.

1.4. ~~Solution~~Data Model Overview

The ~~proposed~~ data model splits the design into ~~2~~ two YANG modules: ~~-~~ (1) Traffic Generator module (TG~~),~~) and (2) Traffic Analyzer module (TA). The modules are implemented as augmentations of the "ietf-interfaces" [RFC8343] module adding configuration and state data that model~~s~~ the functionality of a network interconnect tester. The TA and TG modules concept is illustrated with the ~~following~~ diagram depicted in Figure 2 ~~of~~ with a tester with two interfaces (named "eth0" and "eth1") connected in a loop with single DUT~~:~~.

```
                  +----------------+
       eth0.egress |                | eth1.ingress
      +------------| TG  tester  TA |<-------------+
      |            |                |              |
      |            +----------------+              |
      |                                            |
      |              +------------+                |
      |              |            |                |
      +------------->|    DUT     |----------------+
                     |            |
                     +------------+
```

Figure 2: TBC

2. Using the N~~n~~etwork ~~interconnect~~ Interconnect ~~tester~~ Tester ~~model~~Data Model

A ~~Basic~~ basic example of how the data model can be used in transactional network test program to control the testers part of a network and report counter statistics and timing measurement data is presented in Appendix A. All example cases present the configuration and state data from a single test trial. One of the examples demonstrates the use of the [RFC2544] defined test frame packet.

The search algorithm logic that operates to control the trial configuration is outside the scope of this document.

~~One of the examples demonstrates the use of the [RFC2544] defined testframe packet.~~

Commenté [MB15]: Is it possible to use the modules in a separate mode where TG and TA are embeded in distinct entities no colocated?

Commenté [MB16]: Please use «test frame» instead of «testframe» for consistency with RFC2544

3. Traffic Generator Module ~~Tree Diagram~~Description

TBC

```
module: ietf-traffic-generator

  augment /if:interfaces/if:interface:
    +--rw traffic-generator!
       +--rw (type)
       |  +--:(single-stream)
       |  |  +--rw testframe-type?     identityref
       |  |  +--rw frame-size          uint32
       |  |  +--rw frame-data?         binary
       |  |  +--rw interframe-gap      uint32
       |  |  +--rw interburst-gap?     uint32
       |  |  +--rw frames-per-burst?   uint32
       |  |  +--rw modifiers
       |  |     +--rw modifier* [id]
       |  |        +--rw id             uint32
       |  |        +--rw action         identityref
       |  |        +--rw offset         uint32
       |  |        +--rw mask           binary
       |  |        +--rw repetitions    uint32
       |  +--:(multi-stream)
       |     +--rw streams
       |        +--rw stream* [id]
       |           +--rw id                   uint32
       |           +--rw testframe-type?      identityref
       |           +--rw frame-size           uint32
       |           +--rw frame-data?          binary
       |           +--rw interframe-gap       uint32
       |           +--rw interburst-gap?      uint32
       |           +--rw frames-per-burst?    uint32
       |           +--rw frames-per-stream    uint32
       |           +--rw interstream-gap      uint32
       |           +--rw modifiers
       |              +--rw modifier* [id]
       |                 +--rw id             uint32
       |                 +--rw action         identityref
       |                 +--rw offset         uint32
       |                 +--rw mask           binary
       |                 +--rw repetitions    uint32
       +--rw realtime-epoch?
       |        yang:date-and-time {realtime-epoch}?
       +--rw start-delay?              uint64
       +--rw total-frames?             uint64
```

4. Traffic Analyzer Module ~~Tree Diagram~~Description

TBC

```
module: ietf-traffic-analyzer

  augment /if:interfaces/if:interface:
    +--rw traffic-analyzer!
       +--rw testframe-filter! {testframe-filter}?
       |  +--rw type      identityref
       |  +--rw mask?     binary
```

Commenté [MB17]: Per RFC9907, we need some narrative text to describe the module. We don't need the description to be verbose but help readers walk through the module. See for example rfc9833

Many of these parameters are anchored in RFC2544. Please add references to the extact section where these are defined.

Commenté [MB18]: Inter-frame-gap would be better

Commenté [MB19]: Inter-burst-gap would be better

Commenté [MB20]: Per RFC9907, we need some narrative text to describe the module. We don't need the description to be verbose but help readers walk through the module. See for example rfc9833

```
      |  +--rw data?     binary
      |  +--rw offset    uint32
      +--rw capture! {capture}?
      |  +--rw start-trigger!
      |  |  +--rw (start-trigger)
      |  |     +--:(frame-index)
      |  |     |  +--rw frame-index?       uint64
      |  |     +--:(testframe-index)
      |  |        +--rw testframe-index?   uint64
      |  +--rw stop-trigger!
      |     +--rw (stop-trigger)
      |        +--:(when-full)
      |           +--rw when-full?   empty
      +--ro state
         +--ro pkts?              yang:counter64
         +--ro octets?            yang:counter64
         +--ro idle-octets?       yang:counter64 {idle-octets-counter}?
         +--ro errors?            yang:counter64
         +--ro testframe-stats
         |  +--ro pkts?                  yang:counter64
         |  +--ro sequence-errors?       yang:counter64
         |  +--ro payload-errors?        yang:counter64
         |  +--ro latency
         |  |  +--ro samples?   uint64
         |  |  +--ro min?       uint64
         |  |  +--ro max?       uint64
         |  |  +--ro average?   uint64
         |  |  +--ro latest?    uint64
         |  +--ro last-sequence-error
         |     +--ro timestamp?   yang:date-and-time
         |     +--ro expected?    uint64
         |     +--ro received?    uint64
         +--ro capture {capture}?
            +--ro frame* [sequence-number]
               +--ro sequence-number    uint64
               +--ro timestamp?         yang:date-and-time
               +--ro length?            uint32
               +--ro data?              binary
```

Commenté [MB21]: Shoudln't there be a data node to track the discontinuity time?

5. Traffic Generator Module YANG

This module imports modules defined in [RFC8343] and [RFC9911].

a mis en forme : Anglais (États-Unis)

```
   <CODE BEGINS> file "ietf-traffic-generator@2026-04-07.yang"

  module ietf-traffic-generator {
    yang-version 1.1;
    namespace "urn:ietf:params:xml:ns:yang:ietf-traffic-generator";
    prefix nttg;

    import ietf-interfaces {
      prefix if;
      reference
        "RFC 8343: A YANG Data Model For for Interface Management";
    }
    import ietf-yang-types {
      prefix yang;
      reference
```

Commenté [MB22]: To match the exact title used in that RFC

```
    "RFC 9911: Common YANG Data Types";
}

organization
  "IETF Benchmarking Methodology Working Group";
contact
  "WG Web:   <http://tools.ietf.org/wg/bmwg/>
   WG List:  BMWG <mailto:bmwg@ietf.org>

   EditorAuthor:   Vladimir Vassilev
                <mailto:vladimir@lightside-instruments.com>";
description
  "This module contains a collection of YANG definitions for
   description and management of network interconnect testers.

   Copyright (c) 2024-2026 IETF Trust and the persons identified as
   authors of the code.  All rights reserved.

   Redistribution and use in source and binary forms, with or
   without modification, is permitted pursuant to, and subject to
   the license terms contained in, the Revised BSD License set
   forth in Section 4.c of the IETF Trust's Legal Provisions
   Relating to IETF Documents
   (https://trustee.ietf.org/license-info).

   All revisions of IETF and IANA published modules can be found
   at the YANG Parameters registry group
   (https://www.iana.org/assignments/yang-parameters).

   This version of this YANG module is part of RFC XXXX; see
   the RFC itself for full legal notices.";
   This version of this YANG module is part of RFC XXXX; see
   the RFC itself for full legal notices.";

revision 2026-04-07 {
  description
    "Initial revision.";

  reference
    "RFC XXXX: A YANG Data Model for
     Network Tester Management";
}

feature multi-stream {
  description
    "The device can generate multi-stream traffic.";
}

feature realtime-epoch {
  description
    "The device can generate traffic precisely
     at configured realtime epoch.";
}

identity testframe-type {
  description
    "Base identity for all test frametestframe types.";
}
```

Commenté [MB23]: This is generic and does not reflect that the module is for the generator part.

Commenté [MB24]: Follow the IETF template in RFC9907

```
identity static {
  base testframe-type;
  description
    "Identity Indicates for static test frametestframe.
     The frame data and size are constant.";
}

identity dynamic {
  base testframe-type;
  description
    "Base Identity identity to be used as base for dynamic
     testframe type identities defined
     in external modules.

     When used itself, it identifies dynamic test frametestframe
     where the last 18 octets of the payload contain
     incrementing sequence number field (8 octets)
     followed by timestamp field in the
     IEEE 1588-2008 format (10 octets). If frame data is defined
     for the last 18 octets of the payload, it will be ignored
     and overwritten with dynamic data according to this
     specification.";
}

identity modifier-action-type {
  description
    "Base identity for all modifier action types.";
}

identity increment {
  base modifier-action-type;
  description
    "Identity forIndicates increment modifier action.";
}

identity decrement {
  base modifier-action-type;
  description
    "Identity forIndicates decrement modifier action.";
}

identity random {
  base modifier-action-type;
  description
    "Identity Indicatesfor random modifier action.";
}

grouping common-data {
  description
    "Common configuration data.";
  leaf realtime-epoch {
    if-feature "realtime-epoch";
    type yang:date-and-time;
    description
      "If this leaf is present, the stream generation will start
       at the specified realtime epoch.";
  }
```

Commenté [MB25]: I appreciate this an example, but can we have a informative reference statement to back this?

a mis en forme : Anglais (États-Unis)

a mis en forme : Anglais (États-Unis)

```
      leaf start-delay {
        type uint64;
        units "octets";
        description
          "If this leaf is present, the traffic generation will start
           after the specified number of idle octets have elapsed.";
      }
      leaf total-frames {
        type uint64;
        description
          "If this leaf is present, the traffic generation will stop
           after the specified number of frames are generated.";
      }
    }

    grouping burst-data {
      description
        "Generated traffic burst parameters.";
      leaf testframe-type {
        type identityref {
          base nttg:testframe-type;
        }
        default "nttg:static";
        description
          "In case of dynamic test_frames this leaf specifies
           the dynamic test_frame identitytype.";
      }
      leaf frame-size {
        type uint32;
        units "octets";
        mandatory true;
        description
          "Size of the frames generated. For example, for
           Ethernet interfaces the following definition
           applies:

           Ethernet frame-size, in octets, includes:
           * Destination Address (6 octets),
           * Source Address (6 octets),
           * Frame Type (2 octets),
           * Data (min 46 octets or 42 octets + 4 octets 802.1Q tag),
           * CRC Checksum (4 octets).

           Ethernet frame-size does not include:
           * Preamble (dependent on MAC configuration
                      by default 7 octets),
           * Start of frame delimiter (1 octet)

           Minimum standard Ethernet frame-size is 64 bytes but
           generators might support smaller sizes for validation.";
      }
      leaf frame-data {
        type binary;
        description
          "The raw frame data.
           The length of the specified data can be
           Shorter, then the '../frame-size' value
           specifying only the header or the header and the
```

Commenté [MB26]: Where Idle octets is defined?

Commenté [MB27]: This does not cover static mention

Commenté [MB28]: Expand

Commenté [MB29]: Can we have a reference statement?

```
            payload with or without the 4 4-byte CRC Checksum
            in the case of a an Ethernet frame. If the frame data string
            specified is longer, then '../frame-size' the excess
            of the specified frame data is trunkatedtruncated.

            In multi-stream mode, if 'frame-data' leaf for one of the
            streams is not specified, it is the value specified for
            the closest stream that precedes it that is used.";
      }
      leaf interframe-gap {
        type uint32;
        units "octets";
        mandatory true;
        description
          "Length of the idle period between generated frames.

            For example, for Ethernet interfaces the following
            definition applies:

            Ethernet interframe-gap between transmission of frames
            known as the inter_frame gap (IFG). A brief recovery time
            between frames allows devices to prepare for
            reception of the next frame. The minimum
            interframe gap is 96 96-bit times (12 12-octet times) (the
time it
            takes to transmit 96 bits (12 octets) of raw data on the
            medium). However, the preamble (7 octets) and start of
            frame delimiter (1 octet) are considered a constant gap that
            should be included in the inter_frame- gap. Thus, the minimum
            value for standard Ethernet transmission should be considered
            20 octets.";
      }
      leaf interburst-gap {
        type uint32;
        units "octets";
        description
          "Is Similar similar to the interframe-gap but takes place
between
            any two bursts of the stream.";
      }
      leaf frames-per-burst {
        type uint32;
        description
          "Indicates Number number of frames contained in a burst.";
      }
    }

    grouping modifier-data {
      description
        "Modifier parameters.";
      container modifiers {
        description
          "Container holding the configured modifiers list.";
        list modifier {
          key "id";
          description
            "Each modifier specifies the action to be performed
             on data at a certain offset.";
```

Commenté [MB30]: To be consistent with «Section 3.7 of rfc1242»

Commenté [MB31]: Can we expand a bit to have a meaningful description?

Commenté [MB32]: Can't be this used for retrieval as well?

```
          leaf id {
            type uint32;
            description
              "Number specifying the identifier of the modifier.";
          }
          leaf action {
            type identityref {
              base nttg:modifier-action-type;
            }
            mandatory true;
            description
              "In case of dynamic test framestestframes, this leaf
specifies
               the dynamic test frame testframe identitytype.";
          }
          leaf offset {
            type uint32;
            units "octets";
            mandatory true;
            description
              "Offset in octets of the modified data of the frame.";
          }
          leaf mask {
            type binary;
            mandatory true;
            description
              "Bit mask of the actual bits affected by the modifier.";
          }
          leaf repetitions {
            type uint32;
            mandatory true;
            description
              "Count of the packets that will repeat the data before
               the modifier makes the next update.";
          }
        }
      }
    }

    grouping multi-stream-data {
      description
        "Multi stream traffic generation parameters.";
      container streams {
        description
          "Non-presenceA container holding the configured stream list.";
        list stream {
          key "id";
          description
            "Each stream repeats a burst until frames-per-stream
             count is reached followed by interstream-gap delay.";
          leaf id {
            type uint32;
            description
              "Number specifying the order of the stream.";
          }
          uses burst-data;
          leaf frames-per-stream {
            type uint32;
```

Commenté [MB33]: The description should also mention the static case.

Commenté [MB34]: Shouldn't a «units» statement be used?

Commenté [MB35]: As this may be used for retrieval as well

```
            mandatory true;
            description
              "The count of frames to be generated before
               generation of the next stream is started.";
          }
          leaf interstream-gap {
            type uint32;
            units "octets";
            mandatory true;
            description
              "Idle period after the last frame of the last burst.";
          }
          uses modifier-data;
        }
      }
    }

    augment "/if:interfaces/if:interface" {
      description
        "Traffic generator augmentations of 'ietf-interfaces'.";
      container traffic-generator {
        presence "Enables the traffic generator.";
        description
          "Traffic generator configuration data.";
        choice type {
          mandatory true;
          description
            "Choice of the type of the data model of the generator:.
             Single or multi stream.";

          case single-stream {
            uses burst-data;
            uses modifier-data;
          }
          case multi-stream {
            uses multi-stream-data;
          }
        }
        uses common-data;
      }
    }


  }

   <CODE ENDS>
```

6. Traffic Analyzer Module YANG

This module imports modules defined in [RFC8343] and [RFC9911].

```
   <CODE BEGINS> file "ietf-traffic-analyzer@2026-04-07.yang"

  module ietf-traffic-analyzer {
    yang-version 1.1;
    namespace "urn:ietf:params:xml:ns:yang:ietf-traffic-analyzer";
    prefix ntta;
```

Commenté [MB36]: Should there be a condition to add the augmentation?

Commenté [MB37]: This can be for retrieval as well

a mis en forme : Anglais (États-Unis)

```
import ietf-interfaces {
  prefix if;
  reference
    "RFC 8343: A YANG Data Model For for Interface Management";
}
import ietf-yang-types {
  prefix yang;
  reference
    "RFC 9911: Common YANG Data Types";
}

organization
  "IETF Benchmarking Methodology Working Group";
contact
  "WG Web:   <http://tools.ietf.org/wg/bmwg/>
   WG List:  BMWG <mailto:bmwg@ietf.org>

   EditorAuthor:   Vladimir Vassilev
              <mailto:vladimir@lightside-instruments.com>";
description
  "This module contains a collection of YANG definitions for
   description and management of network interconnect testers.

   Copyright (c) 2024 2026 IETF Trust and the persons identified as
   authors of the code.  All rights reserved.

   Redistribution and use in source and binary forms, with or
   without modification, is permitted pursuant to, and subject to
   the license terms contained in, the Revised BSD License set
   forth in Section 4.c of the IETF Trust's Legal Provisions
   Relating to IETF Documents
   (https://trustee.ietf.org/license-info).

   All revisions of IETF and IANA published modules can be found
   at the YANG Parameters registry group
   (https://www.iana.org/assignments/yang-parameters).

   This version of this YANG module is part of RFC XXXX; see
   the RFC itself for full legal notices.";
   This version of this YANG module is part of RFC XXXX; see
   the RFC itself for full legal notices.";

revision 2026-04-07 {
  description
    "Initial revision.";
  reference
    "RFC XXXX: A YANG Data Model for
     ___________Network Tester Management";
}

feature testframe-filter {
  description
    "This featureI indicates that the device implements
     a filter that can specify a subset of packets to be
     analyzed as testframes.";
}
```

Commenté [MB38]: Follow the IETF template in RFC9907

```
feature idle-octets-counter {
  description
    "This feature iIndicates that the device implements
     idle-octets counter that accumulates the time
     the link is not utilized. The minimum required
     idle gaps are not counted as idle octets.";
}

feature capture {
  description
    "This feature iIndicates that the device implements
     packet capture functionality.";
}

identity testframe-filter {
  description
    "Base testframe-filter identity.";
}

identity bit-field-match {
  base ntta:testframe-filter;
  description
    "Bit field matching filter. Frames that do not have
     equal values for all the bits of the specified data
     at the offsets specified with bits set in the mask
     are not processed as testframestest frames.";
}

grouping statistics-data {
  description
    "Analyzer statistics.";
  leaf pkts {
    type yang:counter64;
    description
      "Total number of packets analyzed.";
  }
  leaf octets {
    type yang:counter64;
    description
      "This counter is identical with the in-octets/out-octets
       counters defined in RFC8343 except that it counts the
       octets since the analyzer was created.";
  }
  leaf idle-octets {
    if-feature "idle-octets-counter";
    type yang:counter64;
    description
      "Total accumulated period with no frame transmission
       taking place measured in octets at the current link
       speed. Octets not counted in '../octets' but not idle are,
       for example, layer 1 framing octets - for Ethernet interfaces
       7+1 preamble octets per packet.";
  }
  leaf errors {
    type yang:counter64;
    description
      "Count of packets with errors.
       Not counted in the pkts or captured.
```

Commenté [MB39]: «packets» would be bettter.

Commenté [MB40]: May clarify why this is reasoning about packets, not frames, etc.

Commenté [MB41]: This is better highlighted in an Operational Considerations Section

```
      For example, packets with CRC error.";
  }
  container testframe-stats {
    description
      "Statistics for received testframes containing
       either sequence number, payload checksum,
       timestamp or any combination of these features.";
    leaf pkts {
      type yang:counter64;
      description
        "Total count of detected testframes.";
    }
    leaf sequence-errors {
      type yang:counter64;
      description
        "Total count of testframes with
         unexpected sequence number. After each sequence
         error the expected next sequence number is
         updated.";
    }
    leaf payload-errors {
      type yang:counter64;
      description
        "Total count of testframes with
         payload errors.";
    }
    container latency {
      description
        "Latency statistics.";
      leaf samples {
        type uint64;
        description
          "Total count of testframes used for estimating
           the latency statistics. When every testframe is
           used, samples='../pkts.'";
      }
      leaf min {
        type uint64;
        units "nanoseconds";
        description
          "Minimum measured latency.";
      }
      leaf max {
        type uint64;
        units "nanoseconds";
        description
          "Maximum measured latency.";
      }
      leaf average {
        type uint64;
        units "nanoseconds";
        description
          "The sum of all sampled latencies divided
           by the number of samples.";
      }
      leaf latest {
        type uint64;
        units "nanoseconds";
```

Commenté [MB42]: Packet or frames?

```
        description
          "Latency of the latest sample.";
      }
    }
    container last-sequence-error {
      description
        "Last sequence error state data.";
      leaf timestamp {
        type yang:date-and-time;
        description
          "Timestamp of the moment a testframe with
           unexpected sequence number was received.";
      }
      leaf expected {
        type uint64;
        description
          "Expected sequence number.";
      }
      leaf received {
        type uint64;
        description
          "Received sequence number.";
      }
    }
  }
}

grouping capture-config-data {
  description
    "Grouping with a capture configuration container.";
  container capture {
    if-feature "capture";
    presence "Enables the capture functionality";
    description
      "Contains capture parameters.";
    container start-trigger {
      presence "Enables trigger condition that starts
                the capture process.";
      description
        "Configures when the capture start is triggered.";
      choice start-trigger {
        mandatory true;
        description
          "Choice of start-trigger condition.";
        case frame-index {
          description
            "Start capturing frames at the specified frame index.";
          leaf frame-index {
            type uint64;
            description
              "First captured frame index.";
          }
        }
        case testframe-index {
          description
            "Start capturing frames at the specified
             testframe index.";
          leaf testframe-index {
```

```
            type uint64;
            description
              "First captured testframe index.";
          }
        }
      }
    }
    container stop-trigger {
      presence "Enables trigger condition that stops
                the capture process.";
      description
        "Configures when the capture is stopped.";
      choice stop-trigger {
        mandatory true;
        description
          "Choice of stop-trigger condition.";
        case when-full {
          description
            "Stops capturing when the implementation can not store
             more frames.";
          leaf when-full {
            type empty;
            description
              "When present in configuration capture stops when
               the capture buffer is full.";
          }
        }
      }
    }
  }
}

grouping capture-data {
  description
    "Grouping with statistics and data
     of one or more captured frame.";
  container capture {
    if-feature "capture";
    description
      "Statistics and data of
       one or more captured frames.";
    list frame {
      key "sequence-number";
      description
        "Statistics and data of a captured frame.";
      leaf sequence-number {
        type uint64;
        description
          "Incremental counter of frames captured.";
      }
      leaf timestamp {
        type yang:date-and-time;
        description
          "Timestamp of the moment the frame was captured.";
      }
      leaf length {
        type uint32;
        description
```

Commenté [MB43]: Can this be defined? Or at least have a reference statement

```
          "Frame length. Ideally the data captured will be
           of the same length but can be shorter
           depending on implementation limitations.";
      }
      leaf data {
        type binary;
        description
          "Raw data of the captured frame.";
      }
    }
  }
}

grouping filter-data {
  description
    "Grouping with a filter container specifying the filtering
     rules for processing only a specific subset of the
     frames.";
  container testframe-filter {
    if-feature "testframe-filter";
    presence "When present packets are
              filtered before analyzed according
              to the filter type";
    description
      "Contains the filtering rules for processing only
       a specific subset of the frames.";
    leaf type {
      type identityref {
        base ntta:testframe-filter;
      }
      mandatory true;
      description
        "Type of the applied filter. External modules can
         define alternative filter type identities.";
    }
  }
}

augment "/if:interfaces/if:interface" {
  description
    "Traffic analyzer augmentations of 'ietf-interfaces'.";
  container traffic-analyzer {
    presence "Enables the traffic analyzer.";
    description
      "Traffic analyzer for ingress direction.";
    uses filter-data;
    uses capture-config-data;
    container state {
      config false;
      description
        "State data.";
      uses statistics-data;
      uses capture-data;
    }
  }
}

augment "/if:interfaces/if:interface/ntta:traffic-analyzer/"
```

Commenté [MB44]: statistics?

```
          + "ntta:testframe-filter" {
      when "derived-from-or-self(ntta:type, 'ntta:bit-field-match')";
      description
        "Logical AND of masked bit fields.";
      leaf mask {
        type binary;
        description
          "Specifies bit field mask for comparison.
           Non-masked bit fields are ignored.";
      }
      leaf data {
        type binary;
        description
          "Specify data to be matched according to the specified mask.";
      }
      leaf offset {
        type uint32;
        mandatory true;
        description
          "Offset in the frame.";
      }
    }
  }

  <CODE ENDS>
```

7. IANA Considerations

This document registers two URIs and two YANG modules.

7.1. URI Registration

IANA is requested to register the following URIs in the "ns" registry within the "IETF XML Registry" group [RFC3688]:
~~This document registers two URIs in the IETF XML registry [RFC3688]. Following the format in RFC 3688, the following registration is requested to be made:~~

~~URI: urn:ietf:params:xml:ns:yang:ietf-traffic-generator~~
~~URI: urn:ietf:params:xml:ns:yang:ietf-traffic-analyzer~~

~~Registrant Contact: The IESG.~~

~~XML: N/A, the requested URI is an XML namespace.~~
URI: urn:ietf:params:xml:ns:yang:ietf-traffic-generator
Registrant Contact: The IESG
XML: N/A; the requested URI is an XML namespace.

URI: urn:ietf:params:xml:ns:yang:ietf-traffic-analyzer
Registrant Contact: The IESG
XML: N/A; the requested URI is an XML namespace.

Commenté [MB45]: Follow RFC9907 template

7.2. YANG Module Name Registration

IANA is requested to register the following YANG modules in the "YANG Module Names" registry [RFC6020] within the "YANG Parameters" registry group.

~~This document registers two YANG module in the YANG Module Names registry YANG [RFC6020].~~

name: ietf-traffic-generator
Maintained by IANA? Y/N

namespace: urn:ietf:params:xml:ns:yang:ietf-traffic-generator
prefix: nttg
reference: RFC XXXX

name: ietf-traffic-analyzer
Maintained by IANA? Y/N

namespace: urn:ietf:params:xml:ns:yang:ietf-traffic-analyzer
prefix: ntta
reference: RFC XXXX

X. Operational Considerations

8. Security Considerations

This section is modeled after the template described in Section 3.7.1 of [RFC9907].

The YANG modules "ietf-traffic-generator" and "ietf-traffic-analyzer" define a data model that is designed to be accessed via YANG-based management protocols, such as the Network Configuration Protocol (NETCONF) [RFC6241] and RESTCONF [RFC8040]. These YANG-based management protocols (1) have to use a secure transport layer (e.g., Secure Shell (SSH) [RFC4252], TLS [RFC8446], and QUIC [RFC9000]) and (2) have to use mutual authentication.

The Network Configuration Access Control Model (NACM) [RFC8341] provides the means to restrict access for particular NETCONF or RESTCONF users to a preconfigured subset of all available NETCONF or RESTCONF protocol operations and content.

~~The YANG modules defined in this document are designed to be accessed via the NETCONF protocol RFC 6241 [RFC6241]. The lowest NETCONF layer is the secure transport layer and the mandatory to implement secure transport is SSH RFC 6242 [RFC6242]. The NETCONF access control model RFC 8341 [RFC8341] provides the means to restrict access for particular NETCONF users to a pre-configured subset of all available NETCONF protocol operations and content.~~

~~There are a number of data nodes defined in this YANG module which are writable/creatable/deletable (i.e. config true, which is the default). These data nodes may be considered sensitive or vulnerable in some network environments. Write operations (e.g. edit-config) to these data nodes without proper protection can have a negative effect on network operations. These are the subtrees and data nodes and their sensitivity/vulnerability:~~

Commenté [MB46]: Follow template in 3.8.3.1 of RFC9907

Commenté [MB47]: Please add aspects that are import to make use of the model in operation. Some of the consistency checks in the modules can be highlighted here.

Also, I don't find in the modules how to bind by config an egress interface with an ingress interface, when many are supported. Having the assumption and a theory of operation would help. Thanks.

Commenté [MB48]: Please follow the template in 3.7.1 of RFC9907

8.1. ietf-traffic-generator.yang

There are a number of data nodes defined in this YANG module that are writable/creatable/deletable (i.e., "config true", which is the default). All writable data nodes are likely to be sensitive or vulnerable in some network environments. Write operations (e.g., edit-config) and delete operations to these data nodes without proper protection or authentication can have a negative effect on network operations. The following subtrees and data nodes have particular sensitivities/vulnerabilities:

The ietf-traffic-generator YANG module controls a stateless traffic generator which is intended to be used for testing and verification purposes but can be used for malicious purposes like generating network traffic part of a Denial-of-Service (DoS) attack. This should be taken into consideration when granting write access to the following container and descendant data nodes:

* /if:interfaces/if:interface/nttg:traffic-generator

Some of the readable data nodes in this YANG module may be considered sensitive or vulnerable in some network environments. It is thus important to control read access (e.g., via get, get-config, or notification) to these data nodes. Specifically, the following subtrees and data nodes have particular sensitivities/ vulnerabilities:

TBC.

8.2. ietf-traffic-analyzer.yang

Commenté [MB49]: Similar changes as above to follow the 9907 template

The ietf-traffic-analyzer YANG module controls a traffic analyzer which is designed for use in testing and verification but can be used for reading information contained in packets sent and received on any of the interfaces on systems that implement the capture feature. This should be taken into consideration when granting read access to the following container and descendant data nodes:

* /if:interfaces/if:interface/ntta:traffic-analyzer/ntta:capture

9. References

9.1. Normative References

[RFC6020] Bjorklund, M., Ed., "YANG - A Data Modeling Language for the Network Configuration Protocol (NETCONF)", RFC 6020, DOI 10.17487/RFC6020, October 2010, <https://www.rfc-editor.org/info/rfc6020>.

[RFC6241] Enns, R., Ed., Bjorklund, M., Ed., Schoenwaelder, J., Ed., and A. Bierman, Ed., "Network Configuration Protocol (NETCONF)", RFC 6241, DOI 10.17487/RFC6241, June 2011, <https://www.rfc-editor.org/info/rfc6241>.

[RFC6242] Wasserman, M., "Using the NETCONF Protocol over Secure Shell (SSH)", RFC 6242, DOI 10.17487/RFC6242, June 2011, <https://www.rfc-editor.org/info/rfc6242>.

Commenté [MB50]: This is not normative. Please move it to informative.

Commenté [MB51]: Idem as previous one.

This si not needed in the new template BTW

[RFC7224] Bjorklund, M., "IANA Interface Type YANG Module", RFC 7224, DOI 10.17487/RFC7224, May 2014, <https://www.rfc-editor.org/info/rfc7224>.

[RFC7950] Bjorklund, M., Ed., "The YANG 1.1 Data Modeling Language", RFC 7950, DOI 10.17487/RFC7950, August 2016, <https://www.rfc-editor.org/info/rfc7950>.

[RFC8341] Bierman, A. and M. Bjorklund, "Network Configuration Access Control Model", STD 91, RFC 8341, DOI 10.17487/RFC8341, March 2018, <https://www.rfc-editor.org/info/rfc8341>.

[RFC8343] Bjorklund, M., "A YANG Data Model for Interface Management", RFC 8343, DOI 10.17487/RFC8343, March 2018, <https://www.rfc-editor.org/info/rfc8343>.

[RFC9911] Schönwälder, J., Ed., "Common YANG Data Types", RFC 9911, DOI 10.17487/RFC9911, December 2025, <https://www.rfc-editor.org/info/rfc9911>.

Commenté [MB52]: Check this is used.

9.2. Informative References

[IEEE1588] IEEE, "IEEE 1588-2008", 2008.

[IEEE802.3-2014] IEEE WG802.3 - Ethernet Working Group, "IEEE 802.3-2014", 2014.

[RFC2544] Bradner, S. and J. McQuaid, "Benchmarking Methodology for Network Interconnect Devices", RFC 2544, DOI 10.17487/RFC2544, March 1999, <https://www.rfc-editor.org/info/rfc2544>.

[RFC3688] Mealling, M., "The IETF XML Registry", BCP 81, RFC 3688, DOI 10.17487/RFC3688, January 2004, <https://www.rfc-editor.org/info/rfc3688>.

[RFC8340] Bjorklund, M. and L. Berger, Ed., "YANG Tree Diagrams", BCP 215, RFC 8340, DOI 10.17487/RFC8340, March 2018, <https://www.rfc-editor.org/info/rfc8340>.

Commenté [MB53]: This is not listed in the doc. Also, check if this not normative.

Commenté [MB54]: Idem as previous one.

Commenté [MB55]: I think this one should be normative.

Commenté [MB56]: This is normative.

Appendix A. Examples

The ~~following~~ topology depicted in Figure 3 ~~will be~~is used for the examples in this ~~section:~~appendix.

```
+-------------+           +------------+           +------------+
|             | eth0 eth0 |            | eth1 eth0 |            |
| tester0   TG|>--------->|    dut0    |>--------->|TA  tester1 |
|             |           |            |           |            |
+-------------+           +------------+           +------------+
```

Figure 3: TBC.

A.1. Basic Test Program

This pseudo code program orchestrates a network test and shows how

the model can be used:

```
#Connect to network
net=connect("topology.xml")

# Configure DUTs and enable traffic-analyzers
net.node("dut0").edit( \
  "create /interfaces/interface[name='eth0'] -- type=ethernetCsmacd")
net.node("dut0").edit(
  "create /interfaces/interface[name='eth1'] -- type=ethernetCsmacd")
net.node("dut0").edit(
  "create /flows/flow[id='t0'] -- match/in-port=eth0 "
  "actions/action[order='0']/output-action/out-port=eth1")

net.node("tester1").edit(
  "create /interfaces/interface[name='eth0']/traffic-analyzer")
net.commit()

#Get network state - before
before=net.get()

# Start traffic
net.node("tester0).edit(
  "create /interfaces/interface[name='eth0']/traffic-generator -- "
  "frame-size=64 interframe-gap=20")

net.commit()

time.sleep(60)

# Stop traffic
net.node("tester1").edit("delete /interfaces/interface[name='eth0']/"
                         "traffic-generator")
net.commit()

#Get network state - after
after=net.get()

#Report
sent_pkts=delta("tester0",before,after,
  "/interfaces/interface[name='eth0']/statistics/out-unicast-pkts")

received_pkts=delta("tester1",before,after,
  "/interfaces/interface[name='eth0']/statistics/in-unicast-pkts")

latency_max=absolute(after,
  "/interfaces/interface[name='eth0']/traffic-analyzer/state/"
  "testframe-stats/latency/max")

#Cleanup
net.node("tester1").edit(
   "delete /interfaces/interface/traffic-analyzer")
net.node("dut0").edit("delete /flows")
net.node("dut0").edit("delete /interfaces")
net.commit()
```

A.2. Generating RFC2544 Test Frames

Commenté [MB57]: Can we please indicate in the text the language + add an informative reference.

~~In Appendix sec.~~ C.2.6.4 of [RFC2544] ~~Test Frames~~specifies ~~-~~a detailed format ~~is specified~~. The frame-data leaf allows an operator to full control over the generated frames payload.

```
...
net.node("tester1").edit(
  "merge /interfaces/interface[name='eth0']/"
  "traffic-generator -- frame-data="
  "6c:a9:6f:00:00:02:6c:a9:6f:00:00:01:08:00:45:00:"
  "00:2e:d4:a5:00:00:0a:11:58:16:c0:00:02:01:C0:00:"
  "02:02:c0:20:00:07:00:1a:00:00:01:02:03:04:05:06:"
  "07:08:09:0a:0b:0c:0d:0e:0f:10:11:12")
...
```

A.3. Generating F~~f~~rames with ~~dynamic~~ Dynamic ~~data~~ Data ~~fields~~Fields

Adding a modifier to single stream traffic generator configuration. In this example, ~~the~~ the highest 16 bits of the source MAC address of the generated Ethernet frame will be incremented.

```
...
net.node("tester1").edit(
  "merge /interfaces/interface[name='eth0']/"
  "traffic-generator/modifiers/modifier[id='0'] -- "
  "offset=6 mask=ff:ff action=increment repetitions=1")
...
```

A.4. Example: NETCONF <get-config> Reply

This section gives an example of a reply to the NETCONF <get-config> request for <running> for a device that implements the example data models above.

```
<rpc-reply
    xmlns="urn:ietf:params:xml:ns:netconf:base:1.0"
    message-id="101">
  <data>
    <interfaces
        xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces"
        xmlns:ianaift="urn:ietf:params:xml:ns:yang:iana-if-type"
        xmlns:nttg="urn:ietf:params:xml:ns:yang:ietf-traffic-generator">
      <interface>
        <name>eth0</name>
        <type>ianaift:ethernetCsmacd</type>
        <nttg:traffic-generator>
          <frame-size>64</frame-size>
          <frame-data>
              bKlvAAACbKlvAAABCABFAAAu1KUAAAoRWBbAAAIB
              wAACAsAgAAcAGgAAAQIDBAUGBwgJCgsMDQ4PEBES
          </frame-data>
          <interframe-gap>20</interframe-gap>
        </nttg:traffic-generator>
      </interface>
    </interfaces>
  </data>
```

Commenté [MB58]: This should be updated to mention RFC2544

Commenté [MB59]: Maybe add some narrative to explain how to read this

Commenté [MB60]: Please check as this is not a well formatted sentence.

Commenté [MB61]: Which one?

```
</rpc-reply>
```

Author's Address


```
   Vladimir Vassilev
   Lightside Instruments AS
   Email: vladimir@lightside-instruments.com
```